STRING MATCHING ALGORITHM
MODULE-3

String Matching

- String Matching Algorithm is also called "**String Searching Algorithm**." It Finds all occurrences of a pattern in a given text(or body of text).
- Given a text array, T [1.....n], of n character and a pattern array, P [1.....m], of m characters.
- The problems are to find an integer s, called **valid shift** where $0 \leq s < n-m$ and T [s+1.....s+m] = P [1.....m].
- In other words, to find whether P in T, where P is a substring of T. The item of P and T are character drawn from some finite alphabet such as {0, 1} or {A, B .....Z, a, b..... z}.

Algorithms used for String Matching

There are different method to find the string matching

- The Naive String Matching Algorithm
- The Rabin-Karp-Algorithm
- The Knuth-Morris-Pratt Algorithm(KMP)

The Naive String Matching Algorithm

The naïve approach tests all the possible placement of **Pattern P [1.....m] relative to text T [1.....n]**. We try shift s = 0, 1.....n-m, successively and for each shift s. Compare T [s+1.....s+m] to P [1.....m].

The naïve algorithm **finds all valid shifts** using a loop that checks the condition P [1.....m] = T [s+1.....s+m] for each of the n - m +1 possible value of s.

```
NAIVE-STRING-MATCHER (T, P)
1. n ← length [T] //Text
2. m ← length [P] //Pattern
3. for s ← 0 to n -m
4.         do if P [1.....m] = T [s + 1....s + m]
5.                   then print "Pattern occurs with shift" s
```

Analysis: This for loop from 3 to 5 executes for n-m + 1(we need at least m characters at the end) times and in iteration we are doing m comparisons. So the total complexity is O [(n-m+1)*m] → **O(nm)**.

Time Complexity

Analysis:

- This for loop from 3 to 5 executes for n-m + 1(we need at least m characters at the end) times and in iteration we are doing m comparisons. So the total complexity is O [(n-m+1)*m] → **O(nm)**.

Example: The text and pattern is given below and find the index value at which pattern matched with text.

text : a b c d e a b f g f

pattern : e a b

Step 1 : a b c d e a b f g h
e a b ...pattern is not matched.

Step 2 : a b c d e a b f g h
e a b ...pattern is not matched.

Step 3 : a b c d e a b f g h
e a b ...pattern is not matched.

Step 4 : a b c d e a b f g h
e a b ...pattern is not matched.

Step 4 : a b c d e a b f g h
e a b ...pattern is matched.